C# Programlama

Lambda İfadeleri ve LINQ

Emir Öztürk

Lambda İfadesi

KOMUT

=> operatörü

- İsimsiz, satır içi yazılan fonksiyon
- (Parametreler) => ifade
- Parametreler => { durumlar }
- Bir değişkende saklanabilir
  - Func
  - Action
  - Predicate
- Fonksiyon parametre almak için kullanılır

```
(parametreler) => ifade

parametre => ifade

(parametreler) =>
{
    komutlar;
    return deger;
};
```

Lambda İfadesi

ÖRNEK

Metot ve lambda karşılaştırması

Aynı toplama işlemi: isim, dönüş türü ve gövde tek satıra iner

Metot olarak

```
int ToplaFonksiyon(int x, int y)
{
    return x + y;
}
```

Lambda ifadesi olarak

```
Func<int, int, int> Topla = (x, y) => x + y;
```

Lambda İfadesi

ÖRNEK

Yazım biçimleri

```
// Parametresiz
Func<DateTime> Simdi = () => DateTime.Now;

// Tek parametre: parantez zorunlu değil
Func<string, string> Selamla = ad => $"Merhaba {ad}!";

// Birden fazla parametre
Func<string, string, string> TamAd = (ad, soyad) => $"{ad} {soyad.ToUpper()}";

// Gövdeli (çok satırlı) lambda
Func<string, bool> KurumsalEpostaMi = eposta =>
{
    if (string.IsNullOrWhiteSpace(eposta))
        return false;
    return eposta.Contains('@') && eposta.EndsWith(".edu.tr");
};

Console.WriteLine(Selamla("Ayşe"));                          // Merhaba Ayşe!
Console.WriteLine(TamAd("Ayşe", "Yılmaz"));                  // Ayşe YILMAZ
Console.WriteLine(KurumsalEpostaMi("ayse@trakya.edu.tr"));   // True
Console.WriteLine(KurumsalEpostaMi("ayse@gmail.com"));       // False
```

Lambda İfadesi

KOMUT

Func - Action - Predicate

- Func
  - >=0 parametre
  - Dönüş değeri (son tür parametresi)
- Action
  - >=0 parametre
  - Değer döndürmez
- Predicate
  - Bir parametre (çok parametre için tuple)
  - Bool kontroller için

```
Func<T1, T2, ..., TSonuc>      // son tür dönüş türü
Action<T1, T2, ...>            // dönüş yok (void)
Predicate<T>                   // T alır, bool döner
```

Lambda İfadesi

ÖRNEK

Func - Action - Predicate tanımları

```
Func<int, int, int> Topla = (x, y) => x + y;

Action<int,int> ToplaGoster = (x, y) => Console.WriteLine(x + y);

Predicate<int> Topla10danBuyukMu = new Predicate<int>(x=>x > 10);
```

Lambda İfadesi

ÖRNEK

Func

```
// Func<girdi, çıktı>: son tür parametresi dönüş türüdür
Func<double, string> HarfNotu = not =>
    not >= 85 ? "AA" :
    not >= 70 ? "BB" :
    not >= 50 ? "CC" : "FF";

Func<decimal, int, decimal> IndirimliFiyat = (fiyat, yuzde) =>
    fiyat - fiyat * yuzde / 100;

Func<string, string> DosyaAdiTemizle = ad =>
    ad.Trim().ToLower().Replace(' ', '_');

Console.WriteLine(HarfNotu(78));                        // BB
Console.WriteLine(IndirimliFiyat(1250m, 20));           // 1000
Console.WriteLine(DosyaAdiTemizle("  Vize Notları.xlsx")); // vize_notları.xlsx
```

Lambda İfadesi

ÖRNEK

Action

```
// Action<girdi>: değer döndürmez, bir iş yapar
Action<string> Logla = mesaj =>
    Console.WriteLine($"[{DateTime.Now:HH:mm:ss}] {mesaj}");

Action<Ogrenci> KartYazdir = o =>
{
    Console.WriteLine($"{o.Ad} {o.Soyad} ({o.Bolum})");
    Console.WriteLine($"  Not: {o.Not}");
};

Logla("Sistem başlatıldı");      // [09:41:12] Sistem başlatıldı
Logla("Not listesi yüklendi");

// List<T>.ForEach parametre olarak Action<T> alır
ogrenciler.ForEach(KartYazdir);
```

Lambda İfadesi

ÖRNEK

Predicate

Örneklerdeki ogrenciler listesi LINQ bölümünde tanımlanmıştır

```
// Predicate<T>: tek parametre alır, bool döndürür
Predicate<Ogrenci> GectiMi = o => o.Not >= 50;

// List<T> üzerindeki bazı metotlar Predicate<T> bekler
List<Ogrenci> gecenler  = ogrenciler.FindAll(GectiMi);
Ogrenci? ilkKalan       = ogrenciler.Find(o => o.Not < 50);
bool edirneliVarMi      = ogrenciler.Exists(o => o.Sehir == "Edirne");
int bilgisayarIndeksi   = ogrenciler.FindIndex(o => o.Bolum == "Bilgisayar");

Console.WriteLine(gecenler.Count);       // 4
Console.WriteLine(ilkKalan?.Ad);         // Mehmet
Console.WriteLine(edirneliVarMi);        // True
Console.WriteLine(bilgisayarIndeksi);    // 0
```

Lambda İfadesi

KOMUT

Fonksiyonu parametre olarak verme

- Metotlar Func / Action / Predicate türünde parametre alabilir
- Çağıran taraf ne yapılacağını lambda ifadesi olarak gönderir
- Tür parametreleri uyuşmalıdır
- Generic metot ile dönüş türü esnek bırakılabilir

```
static TSonuc Uygula<TSonuc>(int x, int y,
                    Func<int, int, TSonuc> fonksiyon)
{
    return fonksiyon(x, y);
}

Uygula(10, 5, (a, b) => a + b);
```

Lambda İfadesi

ÖRNEK

Tür uyuşmazlığı

UsAl double döndürdüğü için Func<int, int, int> parametresine verilemez

```
class Program {
    3 usages
    public static int Uygula(int x, int y, Func<int, int, int> fonksiyon)
    {
        return fonksiyon(x, y);
    }
    public static void Main(string[] args)
    {
        List<Kisi> kisiListesi = new List<Kisi>();

        Func<int, int, int> Topla = (x, y) => x + y;
        Func<int, int, int> ToplaCokSatir = (x, y) =>
        {
            int sonuc = x + y;
            return sonuc;
        };
        Func<int, int, double> UsAl = (x, y) => Math.Pow(x,y);

        int x = 10;
        int y = 5;
        Console.WriteLine(Uygula(x,y,Topla));
        Console.WriteLine(Uygula(x,y,ToplaCokSatir));
        Console.WriteLine(Uygula(x,y,UsAl));
    }
}
```

Argument type 'System.Func<int,int,double>' is not assignable to parameter type 'System.Func<int,int,int>'

local variable Func<int,int,double> UsAl

Lambda İfadesi

ÖRNEK

Generic çözüm

```
class Program {
    3 usages
    public static T Uygula<T>(int x, int y, Func<int, int, T> fonksiyon)
    {
        return fonksiyon(x, y);
    }
    public static void Main(string[] args)
    {
        List<Kisi> kisiListesi = new List<Kisi>();

        Func<int, int, int> Topla = (x, y) => x + y;
        Func<int, int, int> ToplaCokSatir = (x, y) =>
        {
            int sonuc = x + y;
            return sonuc;
        };
        Func<int, int, double> UsAl = (x, y) => Math.Pow(x,y);

        int x = 10;
        int y = 5;
        Console.WriteLine(Uygula(x,y,Topla));
        Console.WriteLine(Uygula(x,y,ToplaCokSatir));
        Console.WriteLine(Uygula(x,y,UsAl));
    }
}
```

Lambda İfadesi

ÖRNEK

Kendi filtre metodumuz ve hazır metotlar

```
// Ne filtreleneceğini metot değil, çağıran taraf belirler
static List<T> Filtrele<T>(List<T> liste, Func<T, bool> kosul)
{
    List<T> sonuc = new List<T>();
    foreach (T eleman in liste)
        if (kosul(eleman))
            sonuc.Add(eleman);
    return sonuc;
}

List<string> dosyalar = new List<string>
    { "rapor.pdf", "sunum.key", "odev1.cs", "odev2.cs", "ozgecmis.pdf" };

var edirneliler  = Filtrele(ogrenciler, o => o.Sehir == "Edirne");
var pdfler       = Filtrele(dosyalar, d => d.EndsWith(".pdf"));
var kodDosyalari = Filtrele(dosyalar, d => d.EndsWith(".cs"));

// Hazır metotlar da lambda alır: List<T>.Sort(Comparison<T>)
ogrenciler.Sort((a, b) => a.Soyad.CompareTo(b.Soyad));   // soyada göre
ogrenciler.Sort((a, b) => b.Not.CompareTo(a.Not));       // nota göre azalan
```

LINQ

KOMUT

Koleksiyonlar

- Koleksiyonların üzerinde LINQ sorguları kullanılabilir
- Koleksiyonlar sınıf türünden olabilir
- Sınıf türünde olduğunda sorgunun atılacağı element lambda ifadeleri ile belirtilir

```
using System.Linq;

koleksiyon.Metot(eleman => eleman.Alan)
```

LINQ

ÖRNEK

Örneklerde kullanılan veri

```
class Ogrenci
{
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public string Bolum { get; set; }
    public string Sehir { get; set; }
    public double Not { get; set; }
}

List<Ogrenci> ogrenciler = new List<Ogrenci>
{
    new Ogrenci { Ad = "Ayşe",   Soyad = "Yılmaz", Bolum = "Bilgisayar", Sehir = "Edirne",     Not = 92 },
    new Ogrenci { Ad = "Mehmet", Soyad = "Kaya",   Bolum = "Elektrik",   Sehir = "İstanbul",   Not = 45 },
    new Ogrenci { Ad = "Zeynep", Soyad = "Demir",  Bolum = "Bilgisayar", Sehir = "Kırklareli", Not = 78 },
    new Ogrenci { Ad = "Ali",    Soyad = "Çelik",  Bolum = "Makine",     Sehir = "Edirne",     Not = 61 },
    new Ogrenci { Ad = "Elif",   Soyad = "Şahin",  Bolum = "Bilgisayar", Sehir = "Tekirdağ",   Not = 38 },
    new Ogrenci { Ad = "Can",    Soyad = "Arslan", Bolum = "Elektrik",   Sehir = "Edirne",     Not = 85 },
};
```

LINQ

KOMUT

Min - Max - Average

- Temel türden koleksiyonlarda parametresiz kullanılabilir
  - Min()
  - Max()
  - Average()
- Sınıf koleksiyonlarında alan lambda ifadesi ile seçilir
- MinBy / MaxBy değerin kendisini değil, nesneyi döndürür

```
T       Min()
T       Max()
double  Average()

TSonuc  Max(Func<T, TSonuc> secici)
double  Average(Func<T, double> secici)
T       MaxBy(Func<T, TAnahtar> secici)
```

LINQ

ÖRNEK

Min - Max - Average: temel türler

```
// Bir haftalık günlük sıcaklık ölçümleri (°C)
List<double> sicakliklar = new List<double> { 12.5, 14.0, 9.8, 16.2, 11.1, 13.7, 10.4 };

Console.WriteLine($"En düşük : {sicakliklar.Min()} °C");          // 9,8
Console.WriteLine($"En yüksek: {sicakliklar.Max()} °C");          // 16,2
Console.WriteLine($"Ortalama : {sicakliklar.Average():F1} °C");   // 12,5

// Metin koleksiyonunda Min/Max alfabetik sıraya göre çalışır
List<string> sehirler = new List<string> { "Edirne", "Tekirdağ", "Kırklareli" };
Console.WriteLine(sehirler.Min());   // Edirne
```

LINQ

ÖRNEK

Min - Max - Average: sınıf koleksiyonu

```
// Sınıf koleksiyonunda hangi alanın kullanılacağı lambda ile verilir
double enYuksekNot = ogrenciler.Max(o => o.Not);        // 92
double enDusukNot  = ogrenciler.Min(o => o.Not);        // 38
double sinifOrt    = ogrenciler.Average(o => o.Not);    // 66,5

// Değerin kendisi değil, o değere sahip nesne isteniyorsa
Ogrenci birinci = ogrenciler.MaxBy(o => o.Not);         // Ayşe Yılmaz
Ogrenci sonuncu = ogrenciler.MinBy(o => o.Not);         // Elif Şahin

Console.WriteLine($"Sınıf ortalaması: {sinifOrt}");
Console.WriteLine($"Birinci: {birinci.Ad} {birinci.Soyad} ({birinci.Not})");
```

LINQ

KOMUT

Eleman alma

- Koleksiyonlardan belirli elemanları almak için metotlar bulunur
  - First()
  - FirstOrDefault()
  - Last()
  - LastOrDefault()
  - Skip()
  - Take()
- Koşul lambda ifadesi ile verilebilir

```
T                First(Func<T, bool> kosul)
T?               FirstOrDefault(Func<T, bool> kosul)
T                Last(Func<T, bool> kosul)
T?               LastOrDefault(Func<T, bool> kosul)
IEnumerable<T>   Skip(int adet)
IEnumerable<T>   Take(int adet)
```

LINQ

ÖRNEK

First - FirstOrDefault - Last

```
Ogrenci ilk          = ogrenciler.First();                                    // Ayşe
Ogrenci ilkMakine    = ogrenciler.First(o => o.Bolum == "Makine");            // Ali
Ogrenci sonEdirneli = ogrenciler.Last(o => o.Sehir == "Edirne");             // Can

// Koşulu sağlayan eleman yoksa: First → hata, FirstOrDefault → null
Ogrenci? hukukcu = ogrenciler.FirstOrDefault(o => o.Bolum == "Hukuk");
if (hukukcu == null)
    Console.WriteLine("Hukuk bölümünden öğrenci yok.");

Ogrenci hata = ogrenciler.First(o => o.Bolum == "Hukuk");
// InvalidOperationException: Sequence contains no matching element
```

LINQ

ÖRNEK

Skip - Take

```
// Sayfalama: web sitelerindeki "Sonraki sayfa" mantığı
int sayfaBoyutu = 2;
for (int sayfa = 0; sayfa < 3; sayfa++)
{
    var sayfadakiler = ogrenciler.Skip(sayfa * sayfaBoyutu).Take(sayfaBoyutu);
    Console.WriteLine($"Sayfa {sayfa + 1}: " +
        string.Join(", ", sayfadakiler.Select(o => o.Ad)));
}
// Sayfa 1: Ayşe, Mehmet
// Sayfa 2: Zeynep, Ali
// Sayfa 3: Elif, Can

// CSV dosyasında başlık satırını atlamak
var kayitlar = File.ReadAllLines("notlar.csv").Skip(1);
```

LINQ

KOMUT

Select

- Bir metodun koleksiyon üzerinde uygulanması için Select kullanılır
- Metot lambda ifadesi olabilir
- Her elemandan yeni bir değer üretilir
- Dönüş türü IEnumerable<T>

```
IEnumerable<TSonuc> Select(Func<T, TSonuc> donusum)
```

LINQ

Select

ÖRNEK

```
static int Xor1(int x) { return x ^ 1; }

public static void Main()
{
    List<int> sayilar = new List<int>{0,0,1,1,0,1,0,1,1,0,0,1,1,0,1,1,0};
    var sonucFonk :IEnumerable<int> = sayilar.Select(Xor1);
    var sonuc :IEnumerable<int> = sayilar.Select(x :int => x ^ 1);
}
```

LINQ

ÖRNEK

Select: sınıf koleksiyonu

```
// Her öğrenciden tek bir bilgi üret
IEnumerable<string> adSoyadlar = ogrenciler.Select(o => o.Ad + " " + o.Soyad);
// Ayşe Yılmaz, Mehmet Kaya, Zeynep Demir, ...

// Her öğrenciden yeni bir nesne üret (anonim tür)
var durumlar = ogrenciler.Select(o => new
{
    o.Ad,
    Durum = o.Not >= 50 ? "Geçti" : "Kaldı"
});
foreach (var d in durumlar)
    Console.WriteLine($"{d.Ad}: {d.Durum}");   // Ayşe: Geçti, Mehmet: Kaldı, ...

// Lambda yerine hazır bir metot da verilebilir
var dosyaAdlari = Directory.GetFiles("Odevler").Select(Path.GetFileName);
// "Odevler/odev1.cs" → "odev1.cs"
```

LINQ

KOMUT

Where

- Bir koşulun koleksiyon üzerinde kontrolü için Where kullanılır
- Koşulu sağlayan elemanlar döndürülür
- Dönüş türü IEnumerable<T>

```
IEnumerable<T> Where(Func<T, bool> kosul)
```

LINQ

Where

ÖRNEK

```
public static void Main()
{
    List<int> sayilar = new List<int>{0,0,1,1,0,1,0,1,1,0,0,1,1,0,1,1,0};
    var sonucFonk :IEnumerable<int> = sayilar.Where(x :int =>x>0);
    var sonuc :IEnumerable<int> = sayilar.Select(x :int => x ^ 1);
}
```

LINQ

ÖRNEK

Where: sınıf koleksiyonu

```
var bilgisayar = ogrenciler.Where(o => o.Bolum == "Bilgisayar");
// Ayşe, Zeynep, Elif

var edirneliGecenler = ogrenciler.Where(o => o.Sehir == "Edirne" && o.Not >= 50);
// Ayşe, Ali, Can

var kalanlar = ogrenciler.Where(o => o.Not < 50);
Console.WriteLine($"Kalan öğrenci sayısı: {kalanlar.Count()}");   // 2

// Bir klasördeki C# dosyaları
var kodDosyalari = Directory.GetFiles("Proje")
                            .Where(d => Path.GetExtension(d) == ".cs");
```

LINQ

KOMUT

Any ve All

- Verilen koşulun koleksiyon içerisinde en az bir eleman tarafından sağlanma kontrolü için Any
- Verilen koşulun koleksiyon içerisinde tüm elemanlar tarafından sağlandığının kontrolü için All
- Dönüş türü bool

```
bool Any()
bool Any(Func<T, bool> kosul)
bool All(Func<T, bool> kosul)
```

LINQ

Any ve All

ÖRNEK

```
List<int> sayilar = new List<int>{0,0,1,1,0,1,0,1,1,0,0,1,1,0,1,1,0};
var sonucAny :bool = sayilar.Any(x :int =>x>0);
var sonucAll :bool = sayilar.All(x :int =>x>0);
```

LINQ

ÖRNEK

Any ve All: sınıf koleksiyonu

```
bool kalanVar      = ogrenciler.Any(o => o.Not < 50);              // True
bool hepsiGecti    = ogrenciler.All(o => o.Not >= 50);             // False
bool makineVar     = ogrenciler.Any(o => o.Bolum == "Makine");     // True
bool hepsiTrakyali = ogrenciler.All(o => o.Sehir != "Ankara");     // True

// Parametresiz Any: koleksiyon boş mu?
if (!ogrenciler.Any())
    Console.WriteLine("Listede öğrenci yok.");

// Kayıt formu: kullanıcı adı daha önce alınmış mı?
List<string> kullanicilar = new List<string> { "ayse.y", "mkaya", "zdemir" };
string yeniAd = "mkaya";
if (kullanicilar.Any(k => k == yeniAd))
    Console.WriteLine("Bu kullanıcı adı alınmış.");
```

LINQ

KOMUT

Order By

- Sıralama işlemleri için
  - OrderBy
  - OrderByDescending
- İkincil sıralama için
  - ThenBy
  - ThenByDescending
- Sınıf koleksiyonlarında sıralama alana göre lambda ifadesi ile yapılabilir
- Dönüş türü IOrderedEnumerable<T>

```
IOrderedEnumerable<T> OrderBy(Func<T, TAnahtar> anahtar)
IOrderedEnumerable<T> OrderByDescending(Func<T, TAnahtar> anahtar)
IOrderedEnumerable<T> ThenBy(Func<T, TAnahtar> anahtar)
IOrderedEnumerable<T> ThenByDescending(Func<T, TAnahtar> anahtar)
```

LINQ

Order By

ÖRNEK

```
List<Kisi> kisiler = new List<Kisi>();
var adaGore :IOrderedEnumerable<Kisi> = kisiler.OrderBy(x :Kisi => x.Ad);
var soyadaGoreAzalan :IOrderedEnumerable<Kisi> = kisiler.OrderByDescending(x :Kisi => x.Soyad);
```

LINQ

ÖRNEK

Order By ve ThenBy

```
// Not listesi: en yüksekten en düşüğe
var basariSirasi = ogrenciler.OrderByDescending(o => o.Not);
// Ayşe 92, Can 85, Zeynep 78, Ali 61, Mehmet 45, Elif 38

// Önce bölüme göre, aynı bölümde nota göre azalan
var bolumListesi = ogrenciler.OrderBy(o => o.Bolum)
                             .ThenByDescending(o => o.Not);
foreach (var o in bolumListesi)
    Console.WriteLine($"{o.Bolum,-11} {o.Ad,-7} {o.Not}");
// Bilgisayar  Ayşe    92
// Bilgisayar  Zeynep  78
// Bilgisayar  Elif    38
// Elektrik    Can     85
// ...
```

LINQ

KOMUT

Lambda ifadelerini birleştirme

- Lambda ifadeleri art arda çağırılabilir
- Önce seçme, sonra sıralama, daha sonra belirli sayıda alma gibi bir işlem gerçekleştirilebilir
- Her metot bir öncekinin döndürdüğü koleksiyon üzerinde çalışır

```
kaynak.Where(...)        // IEnumerable<T>
      .OrderBy(...)      // IOrderedEnumerable<T>
      .Take(...)         // IEnumerable<T>
      .Select(...)       // IEnumerable<TSonuc>
      .ToList();         // List<TSonuc>
```

LINQ

ÖRNEK

Lambda ifadelerini birleştirme

```
List<Kisi> kisiler = new List<Kisi>();
var ilk5Listesi:List<string> = kisiler.Where(x:Kisi => x.Ad.StartsWith("A"))// IEnumerable<Kisi>
    .OrderBy(x:Kisi => x.Ad)// IOrderedEnumerable<Kisi>
    .Take(5)// IEnumerable<Kisi>
    .Select(x:Kisi =>x.Ad+" "+x.Soyad)// IEnumerable<string>
    .ToList();
```

LINQ

ÖRNEK

Gerçek senaryo: e-ticaret filtresi

```
class Urun
{
    public string Ad { get; set; }
    public string Kategori { get; set; }
    public decimal Fiyat { get; set; }
    public double Puan { get; set; }
    public int Stok { get; set; }
}
// Senaryo: e-ticaret sitesinde filtre
// "Stokta olan, puanı 4 ve üzeri dizüstü bilgisayarlardan en ucuz 5 tanesi
//  (fiyatı aynı olanlarda puanı yüksek olan önce)"
```

LINQ

ÖRNEK

Gerçek senaryo: LINQ ile

Filtrele, sırala, ilk 5'i al, metne dönüştür

```
List<string> oneriler = urunler
    .Where(u => u.Kategori == "DizüstÜ" && u.Stok > 0 && u.Puan >= 4.0)
    .OrderBy(u => u.Fiyat).ThenByDescending(u => u.Puan)
    .Take(5)
    .Select(u => $"{u.Ad} - {u.Fiyat:N0} TL ({u.Puan})")
    .ToList();

// Dell Inspiron 15 - 27.999 TL (4,5)
// Asus Vivobook 16 - 27.999 TL (4,2)
// Huawei MateBook D16 - 29.999 TL (4,3)
// Lenovo IdeaPad 5 - 32.999 TL (4,4)
// Monster Abra A5 - 34.999 TL (4)
```

LINQ

ÖRNEK

Gerçek senaryo: LINQ olmadan (aynı sonuç)

Filtreleme ve sıralama

```
// 1) Filtreleme
List<Urun> uygunlar = new List<Urun>();
foreach (Urun u in urunler)
{
    if (u.Kategori == "Dizüstü" && u.Stok > 0 && u.Puan >= 4.0)
    {
        uygunlar.Add(u);
    }
}

// 2) Sıralama (fiyat artan, eşitse puan azalan)
for (int i = 0; i < uygunlar.Count - 1; i++)
{
    for (int j = 0; j < uygunlar.Count - 1 - i; j++)
    {
        Urun a = uygunlar[j];
        Urun b = uygunlar[j + 1];
        bool yerDegistir = a.Fiyat > b.Fiyat ||
                           (a.Fiyat == b.Fiyat && a.Puan < b.Puan);
        if (yerDegistir)
        {
            uygunlar[j] = b;
            uygunlar[j + 1] = a;
        }
    }
}
```

İlk 5'i alma ve metne dönüştürme

```
// 3) İlk 5'i alma ve 4) metne dönüştürme
List<string> oneriler = new List<string>();
int adet = uygunlar.Count < 5 ? uygunlar.Count : 5;
for (int i = 0; i < adet; i++)
{
    Urun u = uygunlar[i];
    oneriler.Add($"{u.Ad} - {u.Fiyat:N0} TL ({u.Puan})");
}
```

LINQ

KOMUT

LINQ sözdizimi

- Lambda ifadeleri yerine LINQ sözdizimi de kullanılabilir
- SQL'e benzer anahtar kelimeler: from, where, orderby, select
- Derleyici sorguyu metot çağrılarına çevirir

```
from eleman in kaynak
where kosul
orderby anahtar [descending]
select ifade
```

LINQ

LINQ sözdizimi

ÖRNEK

```
List<Kisi> kisiler = new List<Kisi>();
var sonuc :List<string> = (from x :Kisi in kisiler
        where x.Ad.StartsWith("A")
        orderby x.Ad
        select x.Ad + " " + x.Soyad
    ).Take(5).ToList();
```

LINQ

ÖRNEK

LINQ sözdizimi: e-ticaret sorgusu

```
// Aynı e-ticaret sorgusu LINQ sorgu sözdizimi ile
List<string> oneriler = (from u in urunler
                         where u.Kategori == "DizüstÜ" && u.Stok > 0 && u.Puan >= 4.0
                         orderby u.Fiyat, u.Puan descending
                         select $"{u.Ad} - {u.Fiyat:N0} TL ({u.Puan})")
                        .Take(5)
                        .ToList();
```